OBJECT ORIENTED PROGRAMMING (JAVA)

SOLVED PREVIOUS-YEAR QUESTION BANK

B.Tech Computer Engineering | Semester III | D Y Patil University

Exam-ready answers based on the uploaded lecture-slide answers and four analysed previous-year papers.

Theory answers + Java programs + PYQ references

HOW TO USE THIS QUESTION BANK

Every topic is written as an actual exam question, followed immediately by a model answer. Coding questions include complete Java programs intended to be readable and reproducible in an exam. Where the same question appeared with slightly different wording across papers, the references are listed together and one complete answer covers the shared requirement.

PRIORITY ORDER

MUST DO	**Exception handling; OOP concepts; threads and multithreading; inheritance; String and StringBuffer**
HIGH	AWT/Swing; overloading and overriding; abstract class/interface; graphics; recursion; JVM; arrays
MEDIUM	Constructor overloading; multilevel inheritance sphere; access specifiers; argument passing
LOW	Rectangle; sum of first n numbers; nCr/nPr; static members; packages; type casting; palindrome

PAPER KEY

P1 = 5 June 2025 | P2 = 24 November 2025 | P3 = 19 November 2025 (100 marks) | P4 = 26 November 2024

Answer-writing tip: for a 4-5 mark theory question, write the definition and 4-6 clear points. For a 10-mark program, write a brief algorithm/idea, complete code, and sample output or explanation where useful.

A. EXCEPTION HANDLING

Priority: MUST DO

Q1. With a suitable example, explain exception handling in Java.

5 marks | PYQ reference: P4 Q1e; related: P1 Q3c, P2 Q3b, P3 Q1d/Q5c, P4 Q3a

Answer

An exception is an error condition that occurs while a program is running. When it occurs, Java creates an exception object and transfers control to a matching catch block. Exception handling prevents abnormal termination and allows the program to respond to the error.

Java commonly uses three keywords:

- try: contains statements that may cause an exception.
- catch: handles the exception of the specified type.
- finally: contains cleanup code that runs after try/catch in normal execution.

A try block must be followed by at least one catch block or a finally block. A catch block cannot appear by itself.

Java program:

```
class Main {
   public static void main(String[] args) {
      try {
         int divisor = 0;
         int result = 10 / divisor;
         System.out.println(result);
      } catch (ArithmeticException e) {
         System.out.println("Cannot divide by zero");
      } finally {
         System.out.println("Program finished");
      }
   }
}
```

Note / expected output: *The first print is skipped because division by zero throws ArithmeticException. The catch block handles it, and the finally block then runs.*

Q2. Write a Java program to divide two numbers accepted from the user. Handle NumberFormatException and division by zero.

10 marks | PYQ reference: P1 Q3c

Answer

The program reads both values as text and converts them to integers. NumberFormatException handles input that is not a valid integer; ArithmeticException handles a zero divisor.

Java program:

```
import java.io.*;

public class DivideDemo {
   public static void main(String[] args) throws IOException {
      BufferedReader br = new BufferedReader(
         new InputStreamReader(System.in));

      try {
         System.out.print("Enter first number: ");
         int a = Integer.parseInt(br.readLine());
         System.out.print("Enter second number: ");
```

```
            int b = Integer.parseInt(br.readLine());
            System.out.println("Result = " + (a / b));
        } catch (NumberFormatException e) {
            System.out.println("Enter valid integers.");
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Note / expected output: *Example: inputs 10 and 2 produce Result = 5. Inputs 10 and 0 enter the ArithmeticException handler. Input abc enters the NumberFormatException handler.*

Q3. What is the significance of the finally block? Can finally exist without try? Explain with an example.

5 marks | PYQ reference: P2 Q2e

Answer

The finally block is used for cleanup operations, such as closing files or releasing resources. It runs after the try/catch sequence whether the try block completes normally or an exception is handled. It is optional.

No. A finally block cannot exist by itself; it must be attached to a try statement. Valid forms include try-catch-finally and try-finally.

Java program:

```
class FinallyDemo {
    public static void main(String[] args) {
        try {
            int divisor = 0;
            int x = 10 / divisor;
            System.out.println(x);
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic exception handled");
        } finally {
            System.out.println("Finally always runs here");
        }
    }
}
```

Q4. Write a Java program demonstrating multiple catch statements.

4 marks | PYQ reference: P3 Q1d

Answer

Multiple catch blocks allow different exception types to be handled separately. Only the first matching catch block runs. When catch types are related by inheritance, the more specific subclass should be placed before the general superclass exception.

Java program:

```
class MultipleCatchDemo {
    public static void main(String[] args) {
        int[] data = {10, 20, 30};
        try {
            int divisor = 0;
            System.out.println(10 / divisor);
            System.out.println(data[5]);
```

```
    } catch (ArithmeticException e) {
      System.out.println("Arithmetic exception: division by zero");
    } catch (ArrayIndexOutOfBoundsException e) {
      System.out.println("Array index is outside the array");
    } catch (Exception e) {
      System.out.println("Other exception: " + e.getMessage());
    }
    System.out.println("Rest of the code executes");
  }
}
```

B. OOP CONCEPTS

Priority: MUST DO

Q5. Describe the features of object-oriented programming methodology.

5 marks | PYQ reference: P1 Q1a; related: P3 Q2a, P4 Q2a

Answer

Object-oriented programming (OOP) organises a program around objects. An object combines data and the methods that operate on that data. Important features are:

- Class: a user-defined blueprint describing data and behaviour.
- Object: a runtime instance of a class.
- Abstraction: showing essential features while hiding implementation details.
- Encapsulation: combining data and methods into one unit and controlling access to the data.
- Inheritance: creating a new class from an existing class to reuse its fields and methods.
- Polymorphism: one name or interface can represent different behaviours, such as overloaded methods.
- Dynamic binding: the method implementation to run can be selected at runtime.
- Message passing: objects communicate by calling methods and passing arguments.

Q6. Compare and contrast structured/procedural programming and object-oriented programming.

5 marks | PYQ reference: P1 Q2a; P3 Q1a

Answer

Procedural programming organises a program around functions and procedures. OOP organises it around objects that combine data and operations.

Point-by-point comparison:

- Approach: procedural programming commonly follows a top-down approach; OOP commonly follows a bottom-up approach.
- Main unit: functions/procedures versus classes/objects.
- Data and operations: often separate in procedural programming; grouped together through encapsulation in OOP.
- Reuse: procedural reuse is commonly through functions; OOP also supports reuse through classes and inheritance.
- Examples from the lecture material: C, FORTRAN and Pascal are procedural examples; Java and C++ support OOP.

Q7. Compare OOP in Java and C++. State major differences.

5 marks | PYQ reference: P2 Q2a

Answer

Java and C++ both support classes, objects, inheritance and polymorphism. Their major differences in the lecture material are:

- Pointers: Java does not expose pointer arithmetic; C++ supports pointers.
- Multiple inheritance: Java classes cannot extend multiple classes, but interfaces can provide multiple inheritance of type; C++ supports multiple class inheritance.
- Memory management: Java provides automatic garbage collection; C++ commonly uses explicit memory management and destructors.
- Operator overloading: Java supports method overloading but not user-defined operator overloading; C++ supports both.
- Platform: Java bytecode runs on a compatible JVM, supporting platform independence; C++ programs are generally compiled for a target platform.
- Execution: Java source is compiled to bytecode and then executed by the JVM; C++ is generally compiled to native machine code.

Q8. Explain the concept of class and object in OOP with a suitable example.

5 marks | PYQ reference: P2 Q2d

Answer

A class is a blueprint or user-defined data type that describes the fields (state) and methods (behaviour) of its objects. An object is a runtime instance of a class. Objects of the same class can store different values in their fields.

For example, Circle is a class, while c1 and c2 are objects. The radius field stores state, and area() calculates behaviour. The new keyword creates the object.

Java program:

```
class Circle {
   double radius;

   Circle(double r) {
      radius = r;
   }

   double area() {
      return Math.PI * radius * radius;
   }
}

class Main {
   public static void main(String[] args) {
      Circle c1 = new Circle(5);
      Circle c2 = new Circle(2);
      System.out.println(c1.area());
      System.out.println(c2.area());
   }
}
```

C. THREADS AND MULTITHREADING

Priority: MUST DO

Q9. Illustrate the life cycle of a thread in Java.

5 marks | PYQ reference: P1 Q1e; P3 Q2e

Answer

The lecture material describes the traditional thread life cycle using five states:

- Newborn: the Thread object has been created, but start() has not been called.
- Runnable: start() has been called; the thread is ready and waiting for processor time.
- Running: the scheduler selects the thread and it executes run().
- Blocked/Waiting: the thread temporarily cannot continue, for example while sleeping or waiting for a condition.
- Dead: the run() method completes and the thread terminates.

Typical transitions: Newborn --start()--> Runnable; Runnable --scheduler--> Running; Running --sleep()/wait()--> Blocked; Blocked --condition met--> Runnable; Running --run() ends--> Dead. The processor scheduler determines the exact order of execution.

Q10. Explain the concept of multithreading in Java. What is a thread, and how is it implemented?

5 marks | PYQ reference: P2 Q2b; P4 Q2e

Answer

A thread is a lightweight unit of execution within a process. Multithreading means executing two or more threads within one program so that different tasks can make progress concurrently. Every Java program begins with a main thread.

Advantages include better responsiveness, improved use of available processor time, and the ability to perform several activities in one program.

Java commonly creates a thread in either of two ways:

1. Extend the Thread class and override run(). Create the object and call start().

2. Implement Runnable, define run(), pass the object to a Thread, and call start().

Calling start() asks the runtime to schedule a new thread; calling run() directly is just an ordinary method call.

Q11. Write a Java program to create a thread that prints numbers from 1 to 10.

5 marks | PYQ reference: P1 Q2e

Answer

The run() method contains the task to be executed by the thread. Calling start() begins the thread.

Java program:

```
class NumberThread extends Thread {
  public void run() {
    for (int i = 1; i <= 10; i++) {
      System.out.println(i);
```

```
        }
    }
}

class ThreadDemo {
    public static void main(String[] args) {
        NumberThread t = new NumberThread();
        t.start();
    }
}
```

Q12. Write a program with two threads: one displays odd numbers 11 to 20 and the other displays even numbers 1 to 10.

10 marks | PYQ reference: P3 Q4c

Answer

Two different Thread subclasses perform the odd-number and even-number tasks. The outputs may be interleaved because the scheduler decides which thread runs first.

Java program:

```
class OddThread extends Thread {
    public void run() {
        for (int i = 11; i <= 20; i += 2) {
            System.out.println("Odd: " + i);
        }
    }
}

class EvenThread extends Thread {
    public void run() {
        for (int i = 2; i <= 10; i += 2) {
            System.out.println("Even: " + i);
        }
    }
}

class TwoThreads {
    public static void main(String[] args) {
        OddThread odd = new OddThread();
        EvenThread even = new EvenThread();
        odd.start();
        even.start();

        try {
            odd.join();
            even.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println("End of main thread");
    }
}
```

D. INHERITANCE

Priority: MUST DO

Q13. Discuss the importance of inheritance. List the types of inheritance in Java and explain them.

5 marks | PYQ reference: P1 Q2d; P3 Q1c; P4 Q2d

Answer

Inheritance is a mechanism in which a child class acquires accessible fields and methods of a parent class. It supports code reuse and allows a child class to add specialised behaviour or override a parent method.

Types commonly illustrated in Java are:

- Single: one child inherits one parent. Example: Dog extends Animal.
- Multilevel: a chain of inheritance. Example: BabyDog extends Dog, and Dog extends Animal.
- Hierarchical: two or more children inherit the same parent. Example: Dog and Cat extend Animal.
- Multiple: one class inherits from more than one parent class. Java does not allow this with classes, but similar multiple inheritance of type can be achieved by implementing multiple interfaces.

For an exam diagram, use: Animal -> Dog -> BabyDog for multilevel, and Animal -> Dog plus Animal -> Cat for hierarchical.

Q14. Explain multiple inheritance with a suitable example in Java.

5 marks | PYQ reference: P2 Q1d

Answer

Multiple inheritance means deriving one class from two or more parent classes. Java does not permit a class to extend two classes because it can create ambiguity when both parents define the same method. Java allows a class to implement multiple interfaces.

Java program:

```
interface Printable {
   void print();
}

interface Showable {
   void show();
}

class Demo implements Printable, Showable {
   public void print() {
      System.out.println("Print method");
   }

   public void show() {
      System.out.println("Show method");
   }
}

class Main {
   public static void main(String[] args) {
      Demo d = new Demo();
      d.print();
      d.show();
```

```
    }
}
```

Note / expected output: *The Demo class implements both interfaces and supplies the required methods.*

E. STRING AND STRINGBUFFER

Priority: MUST DO

Q15. Describe any five methods of the StringBuffer class.

5 marks | PYQ reference: P1 Q2c

Answer

StringBuffer stores a mutable (changeable) sequence of characters. Five useful methods are:

- append(value): adds a value to the end.
- insert(index, value): inserts a value at a specified position.
- reverse(): reverses the character sequence.
- delete(start, end): removes characters from start up to, but not including, end.
- setCharAt(index, ch): replaces the character at the specified index.

Other methods include length(), capacity(), ensureCapacity(), deleteCharAt() and replace().

Java program:

```
class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("java");
        sb.append(" program");
        sb.insert(0, "Learn ");
        sb.setCharAt(0, 'l');
        System.out.println(sb);
        System.out.println(sb.reverse());
    }
}
```

Q16. Explain String and StringBuffer classes with an example.

5 marks | PYQ reference: P2 Q2f

Answer

String represents an immutable sequence of characters. Once a String is created, modifying it produces a new String object. StringBuffer represents a mutable sequence; methods can modify the same buffer. StringBuffer is useful when text is changed repeatedly.

Java program:

```
class StringDemo {
    public static void main(String[] args) {
        String s = "Java";
        s.concat(" Program");
        System.out.println(s);  // Java: original String unchanged

        StringBuffer sb = new StringBuffer("Java");
        sb.append(" Program");
        System.out.println(sb); // Java Program
    }
}
```

Q17. Explain the String class and describe any four of its methods.

5 marks | PYQ reference: P3 Q2c; P4 Q1c

Answer

String is a Java class used to represent text as objects. String objects are immutable. Four methods are:

- length(): returns the number of characters.
- charAt(index): returns the character at an index.
- substring(start, end): returns part of the string from start up to, but not including, end.
- equals(other): compares the contents of two strings.

Other valid examples are indexOf(), lastIndexOf(), concat(), replace(), trim(), toLowerCase() and toUpperCase().

Java program:

```
class StringMethods {
  public static void main(String[] args) {
    String s = "Java Program";
    System.out.println(s.length());
    System.out.println(s.charAt(2));
    System.out.println(s.substring(0, 4));
    System.out.println(s.equals("Java Program"));
  }
}
```

F. AWT, SWING AND GUI

Priority: HIGH

Q18. Differentiate between AWT and Swing. Why is Swing preferred?

5 marks | PYQ reference: P2 Q2c; P3 Q2f; P4 Q1f

Answer

- Platform support: AWT components are platform-dependent; Swing components are more platform-independent.
- Component type: AWT uses heavyweight native components; Swing largely uses lightweight Java components.
- Look and feel: AWT does not provide Swing-style pluggable look and feel; Swing does.
- Component range: AWT has fewer components; Swing provides components such as JTable, JTabbedPane, JScrollPane, JColorChooser and JTree.
- Design: Swing supports a model-view-controller style for many components and is built on top of AWT.

Swing is preferred for richer and more customisable GUIs because it offers more components, a pluggable look and feel, and better support for complex interfaces.

Q19. Write a Java Swing program to create a simple calculator GUI with digits 0-9, +, -, * and /, and a text field for the result.

10 marks | PYQ reference: P2 Q3c

Answer

This calculator uses buttons for digits 0-9 and the four arithmetic operators. The display shows the expression and then its result; C clears the display. It demonstrates JFrame, JTextField, JButton, GridLayout and ActionListener.

Java program:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {
  private final JTextField display = new JTextField();

  Calculator() {
    setTitle("Calculator");
    setLayout(new BorderLayout(5, 5));
    display.setEditable(false);
    add(display, BorderLayout.NORTH);

    JPanel panel = new JPanel(new GridLayout(4, 4, 4, 4));
    String[] labels = {"7", "8", "9", "/",
                       "4", "5", "6", "*",
                       "1", "2", "3", "-",
                       "C", "0", "=", "+"};
    for (String label : labels) {
      JButton button = new JButton(label);
      button.addActionListener(this);
      panel.add(button);
    }
    add(panel, BorderLayout.CENTER);
    setSize(280, 300);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setVisible(true);
  }

  public void actionPerformed(ActionEvent e) {
    String key = e.getActionCommand();
    if (key.equals("C")) {
      display.setText("");
    } else if (key.equals("=")) {
      calculate();
    } else {
      display.setText(display.getText() + key);
    }
  }

  private void calculate() {
    String text = display.getText();
    int pos = -1;
    for (int i = 1; i < text.length(); i++) {
      char c = text.charAt(i);
      if (c == '+' || c == '-' || c == '*' || c == '/') {
        pos = i;
        break;
      }
    }
```

```
        if (pos < 0) return;
        try {
            double x = Double.parseDouble(text.substring(0, pos));
            double y = Double.parseDouble(text.substring(pos + 1));
            char op = text.charAt(pos);
            double answer;
            switch (op) {
                case '+': answer = x + y; break;
                case '-': answer = x - y; break;
                case '*': answer = x * y; break;
                default:
                    if (y == 0) {
                        display.setText("Cannot divide by zero");
                        return;
                    }
                    answer = x / y;
            }
            display.setText(String.valueOf(answer));
        } catch (NumberFormatException ex) {
            display.setText("Invalid expression");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(Calculator::new);
    }
}
```

Q20. Discuss the different layout managers used in Java GUI programming.

4-5 marks | PYQ reference: P3 Q1e; P4 Q2f

Answer

A layout manager controls the position and size of components inside a container. Important layouts from the lecture material are:

- FlowLayout: places components in a row at preferred size; wraps to the next row if required.
- BorderLayout: arranges components in North, South, East, West and Center regions. It is the default layout for JFrame content panes.
- GridBagLayout: a flexible grid layout that allows components to span multiple rows or columns.

Supplementary examples: GridLayout arranges components in equal-sized cells, while CardLayout displays one panel at a time. These two were not among the three layouts highlighted in the uploaded lecture-slide answer.

Q21. Write a short note on the Java Swing class hierarchy.

5 marks | PYQ reference: P3 Q3c

Answer

Component is the base class for graphical user-interface components. Container extends Component and can hold other components. Swing components generally inherit from JComponent.

A top-level container such as JFrame or JDialog contains a root pane (JRootPane), which manages the content pane and other panes. Components are added to the content pane or to containers within it.

Common Swing classes include JLabel (text), JButton (button), JTextField (single-line text), JTextArea (multi-line text), JCheckBox, JRadioButton, JComboBox, JList, JTable, JScrollPane, JOptionPane, JFileChooser, JColorChooser, JSlider and JProgressBar.

G. METHOD OVERLOADING AND OVERRIDING

Priority: HIGH

Q22. Differentiate method overloading and method overriding.

5 marks | PYQ reference: P1 Q1d; P2 Q3a; P3 Q2d

Answer

- Meaning: overloading uses the same method name with different parameter lists in the same class; overriding gives a subclass its own implementation of an inherited method.
- Parameters: overloaded methods must differ in number, types or order of parameters; an overridden method has a compatible signature with the inherited method.
- Polymorphism: overloading is resolved at compile time; overriding supports runtime polymorphism.
- Inheritance: overloading does not require inheritance; overriding requires an inheritance relationship (or interface implementation).
- Return type alone cannot distinguish overloaded methods. An overriding method cannot reduce the visibility of the inherited method.

Q23. Explain method overloading and write a program with overloaded calculateArea() methods for a circle, rectangle and triangle.

10 marks | PYQ reference: P2 Q3a

Answer

Method overloading is defining methods with the same name but different parameter lists. The compiler selects the appropriate method from the arguments used in the call.

Java program:

```
class Area {
   double calculateArea(double r) {
      return Math.PI * r * r;
   }

   double calculateArea(double length, double width) {
      return length * width;
   }

   double calculateArea(double a, double b, double c) {
```

```
        double s = (a + b + c) / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }
}

class Calculate {
    public static void main(String[] args) {
        Area ob = new Area();
        System.out.println("Circle = " + ob.calculateArea(5));
        System.out.println("Rectangle = " + ob.calculateArea(10, 4));
        System.out.println("Triangle = " + ob.calculateArea(3, 4, 5));
    }
}
```

Note / expected output: *The triangle version uses Heron's formula. For sides 3, 4, 5 it returns area 6.*

Q24. Explain method overriding, its usage and rules with an example.

5 marks | PYQ reference: P3 Q2d

Answer

Method overriding occurs when a subclass provides a new implementation of an inherited instance method using a compatible signature. It is used to achieve runtime polymorphism: a superclass reference can invoke the subclass implementation.

Rules: the method name and parameter list must match; the return type must be the same or covariant; visibility cannot be reduced; static methods are hidden rather than overridden; final methods cannot be overridden; constructors are not overridden.

Java program:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}
```

Note / expected output: *Output: Dog barks. The method selected at runtime is the implementation belonging to the actual Dog object.*

H. ABSTRACT CLASS, INTERFACE AND SHAPE PROGRAMS

Priority: HIGH

Q25. Differentiate between an abstract class and an interface.

5 marks | PYQ reference: P2 Q1b

Answer

- Declaration: an abstract class uses the abstract keyword; an interface uses the interface keyword.
- Methods: an abstract class can contain abstract and concrete methods; an interface defines a contract and can include abstract methods as well as supported default/static methods in modern Java.
- Fields/state: an abstract class can have instance variables and constructors; interface fields are implicitly public, static and final.
- Inheritance: a class can extend only one class but can implement multiple interfaces.
- Object creation: neither an abstract class nor an interface can be instantiated directly; a concrete subclass or implementing class is required.

Q26. Write a Java program with abstract class Shape and subclasses Sphere, Cone and Cylinder. Calculate area and volume using method overriding.

10 marks | PYQ reference: P3 Q5b

Answer

Shape declares abstract area() and volume() methods. Each subclass overrides both methods using its own formula.

Java program:

```
abstract class Shape {
    abstract double area();
    abstract double volume();
}

class Sphere extends Shape {
    double r;
    Sphere(double r) { this.r = r; }
    double area() { return 4 * Math.PI * r * r; }
    double volume() { return (4.0 / 3) * Math.PI * r * r * r; }
}

class Cone extends Shape {
    double r, h;
    Cone(double r, double h) { this.r = r; this.h = h; }
    double area() {
        double l = Math.sqrt(r * r + h * h);
        return Math.PI * r * (r + l);
    }
    double volume() { return Math.PI * r * r * h / 3; }
}

class Cylinder extends Shape {
    double r, h;
    Cylinder(double r, double h) { this.r = r; this.h = h; }
    double area() { return 2 * Math.PI * r * (r + h); }
    double volume() { return Math.PI * r * r * h; }
}
```

```
class ShapeDemo {
   public static void main(String[] args) {
      Shape s = new Sphere(3);
      System.out.println("Sphere area=" + s.area()
         + " volume=" + s.volume());
      s = new Cone(3, 4);
      System.out.println("Cone area=" + s.area()
         + " volume=" + s.volume());
      s = new Cylinder(3, 4);
      System.out.println("Cylinder area=" + s.area()
         + " volume=" + s.volume());
   }
}
```

Note / expected output: *For a cone, l is the slant height. The program demonstrates overriding and runtime polymorphism through the Shape reference.*

Q27. Write a program for a university grace-marks application. Keep academic and sports marks separate, calculate total, and invoke methods through a base-class reference and an interface reference.

10 marks | PYQ reference: P4 Q3b

Answer

The Student class stores the roll number, Test stores academic marks, Sports defines the sports grace marks, and Result computes the total. The main method uses a Student reference and a Sports reference to call inherited/interface methods.

Java program:

```
import java.util.Scanner;

class Student {
   int roll;
   Scanner sc = new Scanner(System.in);
   void getRoll() {
      System.out.print("Enter roll number: ");
      roll = sc.nextInt();
   }
}

class Test extends Student {
   int m1, m2;
   void getMarks() {
      System.out.print("Enter two subject marks: ");
      m1 = sc.nextInt();
      m2 = sc.nextInt();
   }
}

interface Sports {
   float SPORTS_MARKS = 5.8f;
   void showSportsMarks();
}

class Result extends Test implements Sports {
   public void showSportsMarks() {
      System.out.println("Sports grace marks = " + SPORTS_MARKS);
   }
   void display() {
      float total = m1 + m2 + SPORTS_MARKS;
```

```
        System.out.println("Roll number = " + roll);
        System.out.println("Academic marks = " + m1 + ", " + m2);
        System.out.println("Sports marks = " + SPORTS_MARKS);
        System.out.println("Total = " + total);
    }
}

class Main {
    public static void main(String[] args) {
        Result r = new Result();
        Student s = r;      // base-class reference
        Test t = r;
        Sports sp = r;      // interface reference
        s.getRoll();
        t.getMarks();
        sp.showSportsMarks();
        r.display();
    }
}
```

Note / expected output: *The sample stores academic marks separately from sports grace marks and includes both in the total.*

I. GRAPHICS PROGRAMS

Priority: HIGH

Q28. Write a graphics Java program to display "All the Best" using a Frame.

5 marks | PYQ reference: P1 Q1f; P3 Q3f

Answer

The Frame subclass creates a window. The paint(Graphics g) method draws the message. A window listener allows the user to close the window.

Java program:

```
import java.awt.*;
import java.awt.event.*;

public class AllTheBest extends Frame {
    AllTheBest() {
        setTitle("Greeting");
        setSize(400, 200);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });
        setVisible(true);
    }

    public void paint(Graphics g) {
        g.setFont(new Font("Arial", Font.BOLD, 30));
        g.setColor(Color.BLUE);
        g.drawString("All the Best", 100, 110);
    }

    public static void main(String[] args) {
        new AllTheBest();
    }
}
```

Q29. Write a Java program to draw a smiling face using Graphics class methods.

10 marks | PYQ reference: P4 Q3c

Answer

The program draws the face with fillOval(), outlines it with drawOval(), creates two eyes with fillOval(), and draws a smile with drawArc().

Java program:

```
import javax.swing.*;
import java.awt.*;

public class Smiley extends JPanel {
   protected void paintComponent(Graphics g) {
      super.paintComponent(g);
      g.setColor(Color.YELLOW);
      g.fillOval(50, 50, 200, 200);
      g.setColor(Color.BLACK);
      g.drawOval(50, 50, 200, 200);
      g.fillOval(105, 110, 25, 35);
      g.fillOval(170, 110, 25, 35);
      g.drawArc(90, 110, 120, 100, 200, 140);
   }

   public static void main(String[] args) {
      JFrame f = new JFrame("Smiley");
      f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      f.add(new Smiley());
      f.setSize(320, 340);
      f.setVisible(true);
   }
}
```

J. RECURSION PROGRAMS

Priority: HIGH

Q30. Write a program to print the factorial of a number using recursion.

5 marks | PYQ reference: P1 Q2b; P3 Q2b

Answer

Recursion occurs when a method calls itself. A recursive method must have a base case to stop further calls.

Java program:

```
import java.util.Scanner;

class Factorial {
   static long fact(int n) {
      if (n == 0 || n == 1) return 1;
      return n * fact(n - 1);
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a non-negative integer: ");
      int n = sc.nextInt();
      if (n < 0) {
         System.out.println("Factorial is not defined for negatives");
```

```
    } else {
      System.out.println("Factorial = " + fact(n));
    }
  }
}
```

Note / expected output: *For input 5, the output is Factorial = 120.*

Q31. Write a program to print the Fibonacci series using recursion.

5 marks | PYQ reference: P4 Q1b

Answer

In the Fibonacci sequence, fib(0) = 0, fib(1) = 1, and each later value is the sum of the previous two values.

Java program:

```
class Fibonacci {
  static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
  }

  public static void main(String[] args) {
    for (int i = 0; i < 8; i++) {
      System.out.print(fib(i) + " ");
    }
  }
}
```

Note / expected output: *Output: 0 1 1 2 3 5 8 13.*

K. JVM AND PLATFORM INDEPENDENCE

Priority: HIGH

Q32. Explain how Java is platform independent.

5 marks | PYQ reference: P2 Q1a

Answer

Java source code is compiled by javac into bytecode stored in a .class file. Bytecode is not tied to one physical processor. A Java Virtual Machine (JVM) for a particular operating system loads and executes that bytecode.

Because compatible JVMs understand the same bytecode format, the same compiled Java program can run on different platforms without recompiling the source for each one. This is commonly described as "Write Once, Run Anywhere." Each platform still needs its own JVM implementation.

Flow: Program.java -> javac compiler -> Program.class bytecode -> JVM -> execution on the operating system.

Q33. Explain the working of the JVM in detail.

5 marks | PYQ reference: P3 Q3a; P4 Q1a

Answer

1. The programmer writes source code in a .java file.

2. The Java compiler (javac) compiles the source into bytecode stored in a .class file.

3. The class loader loads the required class files into the JVM.

4. The bytecode verifier checks bytecode constraints before execution.

5. JVM runtime memory areas hold data: the heap stores objects; thread stacks hold method frames and local variables; other runtime areas store class-level data and execution state.

6. The execution engine runs the bytecode, using an interpreter and, where available, a Just-In-Time (JIT) compiler for frequently executed code.

7. Garbage collection reclaims memory occupied by objects that are no longer reachable.

Thus, the JVM provides the runtime environment that allows Java bytecode to execute on the underlying platform.

L. ARRAY PROGRAMS

Priority: HIGH

Q34. Write a Java program to accept an m x n matrix and display it in natural form.

5 marks | PYQ reference: P1 Q1c

Answer

Use nested loops: the first pair reads matrix values row by row, and the second pair prints them in the same row-and-column order.

Java program:

```
import java.util.Scanner;

class MatrixDisplay {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter rows and columns: ");
      int m = sc.nextInt();
      int n = sc.nextInt();
      int[][] a = new int[m][n];

      System.out.println("Enter matrix elements:");
      for (int i = 0; i < m; i++) {
         for (int j = 0; j < n; j++) {
            a[i][j] = sc.nextInt();
         }
      }

      System.out.println("Matrix:");
      for (int i = 0; i < m; i++) {
         for (int j = 0; j < n; j++) {
            System.out.print(a[i][j] + " ");
         }
         System.out.println();
      }
```

```
    }
}
```

Q35. Write a Java program to calculate the sum and average of array elements with user input.

5 marks | PYQ reference: P3 Q3b; P4 Q2c

Answer

Read the number of elements, store them in an array, add each element to sum, and divide the sum by the number of elements to obtain the average.

Java program:

```
import java.util.Scanner;

class ArrayAverage {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        if (n <= 0) {
            System.out.println("Number of elements must be positive");
            return;
        }

        int[] a = new int[n];
        int sum = 0;
        for (int i = 0; i < n; i++) {
            System.out.print("Enter element: ");
            a[i] = sc.nextInt();
            sum += a[i];
        }
        double average = (double) sum / n;
        System.out.println("Sum = " + sum);
        System.out.println("Average = " + average);
    }
}
```

M. CONSTRUCTOR OVERLOADING

Priority: MEDIUM

Q36. What is constructor overloading? Write a Java program to demonstrate it.

10 marks | PYQ reference: P1 Q3a; P3 Q4a

Answer

A constructor is a special class member used to initialise objects. It has the same name as the class and has no return type. Constructor overloading means defining more than one constructor in the same class with different parameter lists. Java selects the appropriate constructor based on the arguments supplied when an object is created.

Java program:

```
class Box {
    double width, height, depth;

    Box() {
```

```
    width = height = depth = 1;
  }

  Box(double side) {
    width = height = depth = side;
  }

  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  double volume() {
    return width * height * depth;
  }
}

class BoxDemo {
  public static void main(String[] args) {
    Box b1 = new Box();
    Box b2 = new Box(3);
    Box b3 = new Box(2, 3, 4);
    System.out.println(b1.volume());
    System.out.println(b2.volume());
    System.out.println(b3.volume());
  }
}
```

Note / expected output: *The three objects call three different constructors: no-argument, one-argument and three-argument.*

N. MULTILEVEL INHERITANCE: VOLUME OF A SPHERE

Priority: MEDIUM

Q37. Write a Java program to calculate the volume of a sphere using multilevel inheritance: accept radius in the base class, calculate circle area in the derived class, then calculate volume.

10 marks | PYQ reference: P1 Q3b; P3 Q4b

Answer

The Radius class stores the radius; CircleArea extends Radius and calculates pi r squared; SphereVolume extends CircleArea and calculates volume using V = 4/3 x (pi r squared) x r.

Java program:

```
import java.util.Scanner;

class Radius {
  double r;
  Radius(double r) { this.r = r; }
}

class CircleArea extends Radius {
  double area;
  CircleArea(double r) {
    super(r);
    area = Math.PI * r * r;
  }
}
```

```
class SphereVolume extends CircleArea {
   SphereVolume(double r) { super(r); }
   double volume() {
      return (4.0 / 3) * area * r;
   }
   void display() {
      System.out.println("Radius = " + r);
      System.out.println("Area of circle = " + area);
      System.out.println("Volume of sphere = " + volume());
   }
}

class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter radius: ");
      double r = sc.nextDouble();
      SphereVolume s = new SphereVolume(r);
      s.display();
   }
}
```

Note / expected output: *The inheritance chain is Radius -> CircleArea -> SphereVolume. For radius 3, the volume is approximately 113.10.*

O. ACCESS SPECIFIERS

Priority: MEDIUM

Q38. Explain the different access specifiers in Java.

4-5 marks | PYQ reference: P3 Q1b; P4 Q2b

Answer

Access specifiers control where class members can be accessed.

- private: accessible only inside the declaring class.
- default (no keyword): accessible inside the same package.
- protected: accessible inside the same package and, with inheritance rules, by subclasses outside the package.
- public: accessible from any class where the declaring type is visible.

Private fields are commonly accessed through public getter/setter methods. This protects data and supports encapsulation.

P. ARGUMENT PASSING MECHANISM

Priority: MEDIUM

Q39. Explain the argument passing mechanism with suitable examples. Include the different cases.

5-10 marks | PYQ reference: P2 Q1c; P3 Q5a

Answer

Arguments are values supplied to a method when it is called. The method receives them through parameters. The lecture examples include passing primitive values, passing an instance variable, passing an object and returning an object.

Important Java rule: Java always passes arguments by value. For a primitive value, the method receives a copy of the value, so changing the parameter does not change the caller variable. For an object, the copied value is a reference; the method can modify the object fields through that reference, but assigning the parameter to a different object does not change the caller's reference.

Java program:

```
class PassingDemo {
   static void changePrimitive(int x) {
      x = 99;
   }

   static void changeObject(Box b) {
      b.value = 99;
   }

   public static void main(String[] args) {
      int n = 10;
      changePrimitive(n);
      System.out.println(n); // 10

      Box box = new Box();
      box.value = 10;
      changeObject(box);
      System.out.println(box.value); // 99
   }
}

class Box {
   int value;
}
```

Note / expected output: *The first call changes only the local copy. The second call changes a field in the shared object, so the caller observes the changed field.*

Q. SINGLE-APPEARANCE QUESTIONS

Priority: LOW

Q40. Write a Java program to create a Rectangle class and calculate its area and perimeter.

5 marks | PYQ reference: P1 Q1b

Answer

The Rectangle class stores length and width and provides methods for area and perimeter.

Java program:

```
class Rectangle {
   double length, width;
   Rectangle(double l, double w) {
      length = l;
      width = w;
   }
   double area() { return length * width; }
   double perimeter() { return 2 * (length + width); }
}

class RectDemo {
   public static void main(String[] args) {
      Rectangle r = new Rectangle(5, 3);
      System.out.println("Area = " + r.area());
      System.out.println("Perimeter = " + r.perimeter());
   }
}
```

Note / expected output: *For length 5 and width 3, area = 15 and perimeter = 16.*

Q41. Write a Java program to calculate and display the sum of the first n natural numbers.

5 marks | PYQ reference: P1 Q2f

Answer

The program adds every integer from 1 to n. The formula is n(n+1)/2, but the following loop follows the approach used in the lecture material.

Java program:

```
import java.util.Scanner;

class NaturalSum {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      int sum = 0;
      for (int i = 1; i <= n; i++) {
         sum += i;
      }
      System.out.println("Sum = " + sum);
   }
}
```

Note / expected output: *For n = 10, the sum is 55.*

Q42. Write a Java program to find nCr and nPr.

5 marks | PYQ reference: P2 Q1e

Answer

Permutation: nPr = n! / (n-r)!. Combination: nCr = n! / (r! (n-r)!). The program uses a recursive factorial method.

Java program:

```
import java.util.Scanner;

class CombPerm {
   static long fact(int n) {
      if (n == 0 || n == 1) return 1;
      return n * fact(n - 1);
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n and r: ");
      int n = sc.nextInt();
      int r = sc.nextInt();
      if (n < 0 || r < 0 || r > n) {
         System.out.println("Require n >= r >= 0");
         return;
      }
      long nPr = fact(n) / fact(n - r);
      long nCr = fact(n) / (fact(r) * fact(n - r));
      System.out.println("nPr = " + nPr);
      System.out.println("nCr = " + nCr);
   }
}
```

Note / expected output: *For n = 5 and r = 2, nPr = 20 and nCr = 10. The long type is suitable only for relatively small values because factorials grow quickly.*

Q43. Explain static data members and static methods.

5 marks | PYQ reference: P2 Q1f

Answer

A static member belongs to the class rather than to an individual object. All objects of that class share one static variable. A static method can be called using the class name without creating an object and should directly access static members. A static block executes once when the class is loaded.

Java program:

```
class StaticDemo {
   static int count = 0;
   int value;

   StaticDemo(int v) {
      value = v;
      count++;
   }

   static void showCount() {
      System.out.println("Objects created = " + count);
   }

   public static void main(String[] args) {
      new StaticDemo(10);
      new StaticDemo(20);
```

```
        StaticDemo.showCount();
    }
}
```

Note / expected output: *Output: Objects created = 2.*

Q44. What is a package in Java? Explain its types.

5 marks | PYQ reference: P4 Q1d

Answer

A package is a group of related classes, interfaces and subpackages. Packages organise code, encourage reuse and help prevent naming conflicts.

- Built-in/predefined packages: supplied by the Java library, for example java.lang, java.util, java.io, java.awt and javax.swing. They can be used through import statements.
- User-defined packages: created by the programmer using the package statement, which appears at the beginning of the source file.

Java program:

```
// File: studentpkg/Student.java
package studentpkg;

public class Student {
    public void display() {
        System.out.println("Student package example");
    }
}

// In a separate file outside the package:
import studentpkg.Student;

class PackageDemo {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Note / expected output: *This illustration contains two separate source files; do not save both sections as one Java file. The public class is saved in a file with the same name, inside the folder corresponding to the package.*

Q45. Describe type casting and type conversion in Java.

5 marks | PYQ reference: P3 Q3d

Answer

Type conversion changes a value from one data type to another. Widening conversion converts a smaller compatible type to a larger type and is usually automatic, such as int to double. Narrowing conversion converts a larger type to a smaller type and normally requires an explicit cast; information may be lost.

Java program:

```
class CastDemo {
  public static void main(String[] args) {
    int i = 10;
    double d = i;     // widening: automatic

    double x = 9.78;
    int y = (int) x;   // narrowing: explicit

    System.out.println(d);
    System.out.println(y);
  }
}
```

Note / expected output: *Output: 10.0 and 9. Casting 9.78 to int discards the fractional part.*

Q46. Write a Java program to check whether a string is a palindrome.

5 marks | PYQ reference: P3 Q3e

Answer

A palindrome reads the same forward and backward. Reverse the string and compare the result with the original string.

Java program:

```
import java.util.Scanner;

class Palindrome {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a string: ");
    String s = sc.nextLine();
    String reverse = new StringBuffer(s).reverse().toString();

    if (s.equals(reverse)) {
      System.out.println("Palindrome");
    } else {
      System.out.println("Not a palindrome");
    }
  }
}
```

Note / expected output: *This comparison is case-sensitive and includes spaces. For example, Madam is not treated as a palindrome because M and m differ.*

END OF SOLVED QUESTION BANK

Prepared from the uploaded OOP PYQ frequency analysis and the lecture-slide answer document. Supplementary programs are included where the source answer document identified gaps in slide coverage.